

CSA 0378-DATA STRUCTURES

9. Single linked list- Deletion at Beginning, at End, and at ANY POINT

```
main.c  [Icons]  Share  Run  Output  Clear

1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node
5 {
6     int data;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 // Insert at End (to build the list)
13 void insertEnd(int val)
14 {
15     struct node *newnode = (struct node*)malloc(sizeof(struct node));
16     newnode->data = val;
17     newnode->next = NULL;
18
19     if(head == NULL)
20     {
21         head = newnode;
22         return;
23     }
24
25     struct node *temp = head;
26     while(temp->next != NULL)
27         temp = temp->next;
28
29     temp->next = newnode;
30 }
31
```

```
Enter 5 elements to build the list:
3
5
7
9
0

Initial Linked List: 3 -> 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 1
3 deleted from beginning
Linked List: 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 3
Enter position to delete: 4
0 deleted from position 4
Linked List: 5 -> 7 -> 9 -> NULL

--- Single Linked List Deletion Menu ---
```

```
main.c  [Icons]  Share  Run  Output  Clear

32 // Delete at Beginning
33 void deleteBeginning()
34 {
35     if(head == NULL)
36     {
37         printf("List is empty!\n");
38         return;
39     }
40
41     struct node *temp = head;
42     head = head->next;
43     printf("%d deleted from beginning\n", temp->data);
44     free(temp);
45 }
46
47 // Delete at End
48 void deleteEnd()
49 {
50     if(head == NULL)
51     {
52         printf("List is empty!\n");
53         return;
54     }
55
56     if(head->next == NULL)
57     {
58         printf("%d deleted from end\n", head->data);
59         free(head);
60         head = NULL;
61         return;
62 }
```

```
Enter 5 elements to build the list:
3
5
7
9
0

Initial Linked List: 3 -> 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 1
3 deleted from beginning
Linked List: 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 3
Enter position to delete: 4
0 deleted from position 4
Linked List: 5 -> 7 -> 9 -> NULL

--- Single Linked List Deletion Menu ---
```

```
main.c  [Icons]  Share  Run  Output  Clear

105     }
106     prev = temp;
107     temp = temp->next;
108 }
109
110 prev->next = temp->next;
111 printf("%d deleted from position %d\n", temp->data, pos);
112 free(temp);
113 }
114
115 // Display
116 void display()
117 {
118     struct node *temp = head;
119
120     if(temp == NULL)
121     {
122         printf("List is empty!\n");
123         return;
124     }
125
126     printf("Linked List: ");
127     while(temp != NULL)
128     {
129         printf("%d -> ", temp->data);
130         temp = temp->next;
131     }
132     printf("NULL\n");
133 }
134
135 void main()
136 {
137     return;
138 }
```

```
Enter 5 elements to build the list:
3
5
7
9
0

Initial Linked List: 3 -> 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 1
3 deleted from beginning
Linked List: 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 3
Enter position to delete: 4
0 deleted from position 4
Linked List: 5 -> 7 -> 9 -> NULL

--- Single Linked List Deletion Menu ---
```

```
main.c  [Icons]  Share  Run  Output  Clear

74     printf("%d deleted from end\n", temp->data);
75     free(temp);
76 }
77
78 // Delete at Any Position
79 void deletePosition(int pos)
80 {
81     if(head == NULL)
82     {
83         printf("List is empty!\n");
84         return;
85     }
86
87     struct node *temp = head;
88     struct node *prev = NULL;
89     int i;
90
91     if(pos == 1)
92     {
93         head = temp->next;
94         printf("%d deleted from position %d\n", temp->data, pos);
95         free(temp);
96         return;
97     }
98
99     for(i=1; i<pos; i++)
100     {
101         if(temp == NULL)
102         {
103             printf("Position out of range!\n");
104             return;
```

```
Enter 5 elements to build the list:
3
5
7
9
0

Initial Linked List: 3 -> 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 1
3 deleted from beginning
Linked List: 5 -> 7 -> 9 -> 0 -> NULL

--- Single Linked List Deletion Menu ---
1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
4. Display
5. Exit
Enter choice: 3
Enter position to delete: 4
0 deleted from position 4
Linked List: 5 -> 7 -> 9 -> NULL

--- Single Linked List Deletion Menu ---
```